

(Por qué|Quién|Dónde) Rust

...



Agenda

- Introducción
- Por qué
 - Tipado dinámico/débil
 - Invalidación de iteradores
 - Seguridad de memoria: Use-after-free - Heartbleed
 - Comportamiento indefinido
 - Concurrencia
- Quién
- Dónde



(Por qué|Quién|Dónde) Rust

...

Fortalezas del lenguaje de programación Rust



En esta charla siempre hablamos de *safe* Rust

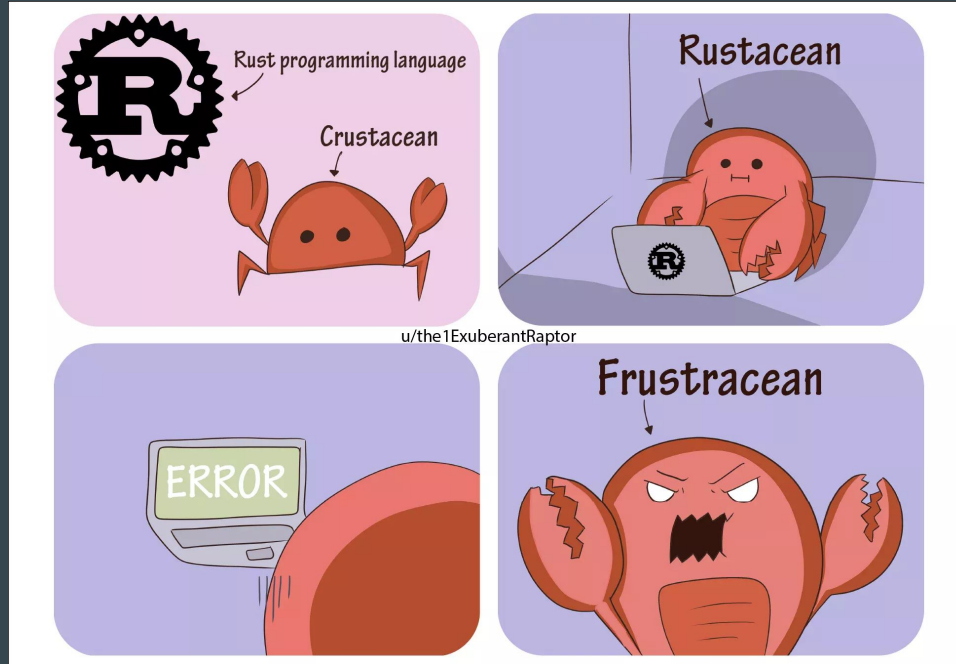
...

Es decir, código que no está encerrado en un bloque *unsafe*



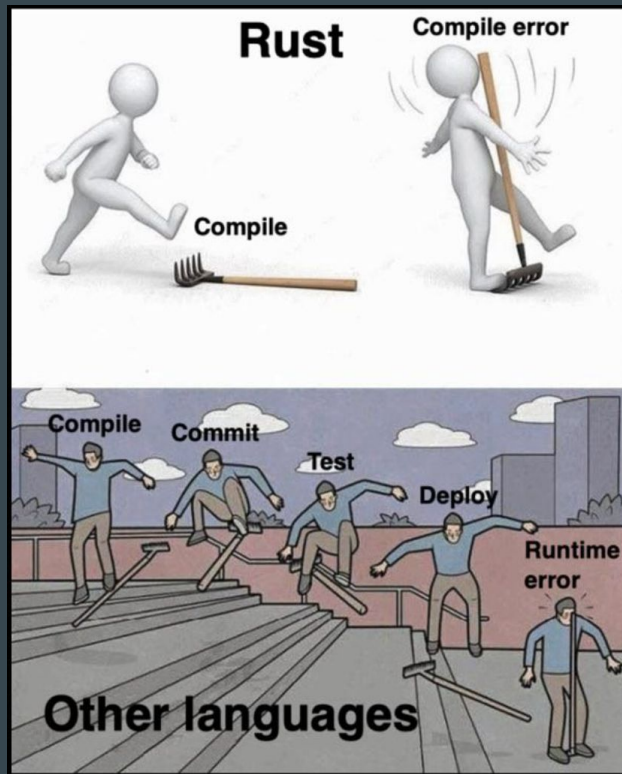
Introducción

Experiencia general con el lenguaje



Introducción

Experiencia general con el lenguaje



Tipado dinámico/débil

...

C, Python, Bash, JavaScript



Tipado dinámico/débil

```
#include <stdio.h>

int calculateSquareArea(int side) {
    return side * side;
}

int main() {
    printf("Area: %i\n", calculateSquareArea(5));
    printf("Area: %i\n", calculateSquareArea('5'));

    return 0;
}
```



Tipado dinámico/débil

```
#include <stdio.h>
```

```
int calculateSquareArea(int side) {  
    return side * side;  
}
```

```
int main() {  
    printf("Area: %i\n", calculateSquareArea(5));  
    printf("Area: %i\n", calculateSquareArea('5'));  
  
    return 0;  
}
```



Terminal

```
$ gcc -Wall area.c -o area
```

```
$ ./area
```

```
Area: 25
```

```
Area: 2809
```



Tipado dinámico/débil

```
#include <stdio.h>
```

```
int calculateSquareArea(int side) {  
    return side * side;  
}
```

```
int main() {  
    printf("Area: %i\n", calculateSquareArea(5));  
    printf("Area: %i\n", calculateSquareArea('5'));  
  
    return 0;  
}
```

5 es un literal de tipo int



Tipado dinámico/débil

```
#include <stdio.h>
```

```
int calculateSquareArea(int side) {  
    return side * side;  
}
```

```
int main() {  
    printf("Area: %i\n", calculateSquareArea(5));  
    printf("Area: %i\n", calculateSquareArea('5'));  
  
    return 0;  
}
```

5 es un literal de tipo int

'5' un literal de tipo char que representa el número 5, cuyo valor numérico es 53.



Tipado dinámico/débil

```
fn calculate_square_area(side: u32) -> u32 {  
    side * side  
}  
  
fn main() {  
    println!("Area: {}", calculate_square_area(5));  
    println!("Area: {}", calculate_square_area('5'));  
}
```



Tipado dinámico/débil

```
fn calculate_square_area(side: u32) -> u32 {  
    side * side  
}  
  
fn main() {  
    println!("Area: {}", calculate_square_area('5'));  
}
```

```
Terminal  
Compiling playground v0.0.1 (/playground)  
error[E0308]: mismatched types  
  --> src/main.rs:7:48  
   |  
7 |         println!("Area: {}", calculate_square_area('5'));  
   |                                     ^^^^^ expected `u32`, found `char`  
   |                                     |  
   |                                     arguments to this function are incorrect  
   |  
note: function defined here  
  --> src/main.rs:1:4  
   |  
1 | fn calculate_square_area(side: u32) -> u32 {  
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
help: you can cast a `char` to a `u32`, since a `char` always occupies 4 bytes  
   |  
7 |         println!("Area: {}", calculate_square_area('5' as u32));  
   |                                                         ++++++  
  
For more information about this error, try `rustc --explain E0308`.
```



Invalidación de iteradores

```
#include <iostream>
#include <list>

void removeEven(std::list<int>& numberList) {
    for (auto it = numberList.begin(); it != numberList.end(); ) {
        if (*it % 2 == 0)
            it = numberList.erase(it);
        ++it;
    }
}

int main() {
    std::list<int> numberList = {2, 2, 8, 1, 10, 6};
    removeEven(numberList);

    for (int n : numberList)
        std::cout << n << " ";

    return 0;
}
```



Invalidación de iteradores

```
#include <iostream>
#include <list>

void removeEven(std::list<int>& numberList) {
    for (auto it = numberList.begin(); it != numberList.end(); ) {
        if (*it % 2 == 0)
            it = numberList.erase(it);
        ++it;
    }
}

int main() {
    std::list<int> numberList = {2, 2, 8, 1, 10, 6};
    removeEven(numberList);

    for (int n : numberList)
        std::cout << n << " ";

    return 0;
}
```

Terminal

```
$ g++ -Wall -Wextra iterator_invalidation.cpp -o ii
$ ./ii
2 1 6
```



Invalidación de iteradores

```
fn remove_even(list: &mut Vec<u8>) {  
    for (i, n) in list.iter().enumerate() {  
        if n % 2 == 0 {  
            list.remove(i);  
        }  
    }  
}  
  
fn main() {  
    remove_even(&mut vec![2, 2, 8, 1, 10, 6])  
}
```



Invalidación de iteradores

```

fn remove_even(list: &mut Vec) {
    error[E0502]: cannot borrow `*list` as mutable because it is also borrowed as immutable
    --> src/main.rs:4:13
    |
    | list.remove(i);
2  |     for (i, n) in list.iter().enumerate() {
    |                                     -----
    |                                     |
    |                                     immutable borrow occurs here
    |                                     immutable borrow later used here
    |
3  |         if n % 2 == 0 {
4  |             list.remove(i);
    |             ^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
    |
    |
}

For more information about this error, try `rustc --explain E0502`.

```



Seguridad de memoria

...

C, C++

Use-after-free

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void greet(char *who) {
    printf("Hello %s\n", who);
    free(who);
}

int main(void) {
    char *who = malloc(4 * sizeof(char));
    strcpy(who, "Foo");

    greet(who);
    printf("Bye %s\n", who);

    return 0;
}
```



Use-after-free

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void greet(char *who) {
    printf("Hello %s\n", who);
    free(who);
}
```

```
int main(void) {
    char *who = malloc(4 * sizeof(char));
    strcpy(who, "Foo");
```

```
    greet(who);
    printf("Bye %s\n", who);
```

```
    return 0;
```

```
}
```

```
Terminal
$ gcc -Wall greet.c -o greet

$ ./greet
Hello Foo
Bye 0*e
```



Use-after-free

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void greet(char *who) {
    printf("Hello %s\n", who);
    free(who);
}
```

```
int main(void) {
    char *who = malloc(4 * sizeof(char));
    strcpy(who, "Foo");
```

```
    greet(who);
    printf("Bye %s\n", who);
```

```
    return 0;
```

```
}
```

Reservamos memoria y
copiamos el string "Foo"



Use-after-free

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void greet(char *who) {
    printf("Hello %s\n", who);
    free(who);
}
```

Greet imprime el mensaje
pero libera la memoria donde
está alojado "Foo"

```
int main(void) {
    char *who = malloc(4 * sizeof(char));
    strcpy(who, "Foo");
```

Reservamos memoria y
copiamos el string "Foo"

```
    greet(who);
    printf("Bye %s\n", who);
```

```
    return 0;
```

```
}
```



Use-after-free

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void greet(char *who) {
    printf("Hello %s\n", who);
    free(who);
}
```

Greet imprime el mensaje pero libera la memoria donde está alojado "Foo".

```
int main(void) {
    char *who = malloc(4 * sizeof(char));
    strcpy(who, "Foo");
```

Reservamos memoria y copiamos el string "Foo".

```
    greet(who);
    printf("Bye %s\n", who);
```

Se intenta utilizar memoria liberada.

```
    return 0;
```

```
}
```



Use-after-free

```
fn greet(who: String) {  
    println!("Hello {who}");  
}  
  
fn main() {  
    let who = String::from("Foo");  
    greet(who);  
    println!("Bye {who}");  
}
```



Use-after-free

```
fn greet(who: String) {  
    println!("Hello {who}");  
    let who = String::from("Foo");  
    --- move occurs because `who` has type `String`, which does not implement the  
    `Copy` trait  
    greet(who);  
}  
  
fn main() {  
    --- value moved here  
    let who = String::from("Foo");  
    println!("Bye {who}");  
    ^^^ value borrowed here after move  
    greet(who);  
}  
  
note: consider changing this parameter type in function `greet` to borrow instead if owning  
the value isn't necessary  
--> src/main.rs:1:15  
1 | fn greet(who: String) {  
  | ----- ^^^^^^ this parameter takes ownership of the value  
  |  
  | in this function  
help: consider cloning the value if the performance cost is acceptable  
7 |     greet(who.clone());  
  |     ++++++++
```



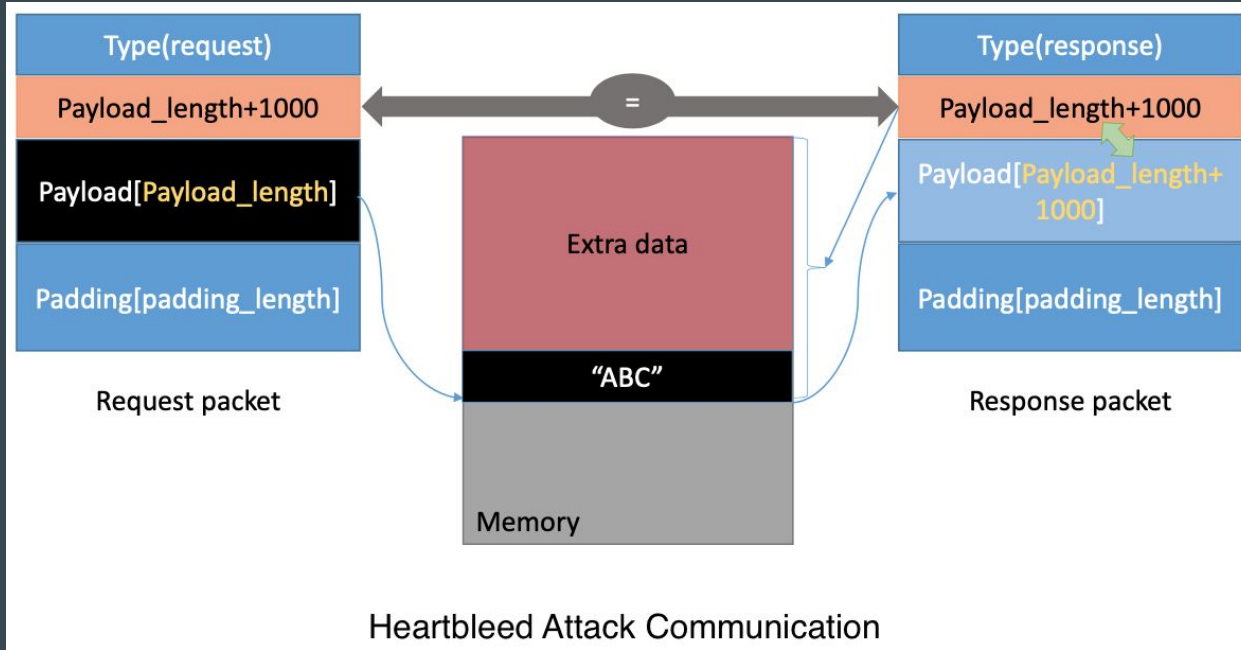
Heartbleed



- En 2014, una vulnerabilidad fue descubierta en OpenSSL.
- Esta vulnerabilidad permitía a un atacante leer sectores de memoria privados del servidor, los cuales potencialmente podrían contener información sobre el usuario o claves privadas.
- Para explotarla, se utilizaba el protocolo **Heartbeat**, que está diseñado para determinar si una conexión segura se mantuvo abierta.



Heartbleed



Heartbleed



...

```
buffer = malloc(request->payload_length)
memcpy(buffer, request->payload_length, request->data);
```

...

1003

“ABC” (solo 3 bytes)



Heartbleed



```
...  
let mut buffer = Vec::new();  
for i in 0..request.payload_length {  
    buffer.push(request.data[i]);  
}  
...
```

Terminal

```
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 3', src/  
heartbleed.rs:6:21  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```



Comportamiento indefinido

...

C, C++



Comportamiento indefinido

Cualquier cosa puede pasar: El estándar no impone ningún requerimiento. El programa puede fallar en tiempo de compilación, o ejecutar de forma incorrecta (ya sea generando resultados incorrecto o saliendo). Incluso, podríamos tener suerte y se podrían generar los resultados esperados.



Comportamiento indefinido

```
#include <iostream>
```

```
int main() {  
    while (1);  
}
```

```
void unreachable() {  
    std::cout << "Hello, world!\n";  
}
```



Comportamiento indefinido

```
#include <iostream>
```

```
int main() {  
    while (1);  
}
```

```
void unreachable() {  
    std::cout << "Hello, world!\n";  
}
```

Terminal

```
$ clang15++ -o main -std=c++17 -O1 -Wall -Wextra main.cpp  
$ ./main  
Hello, world!
```

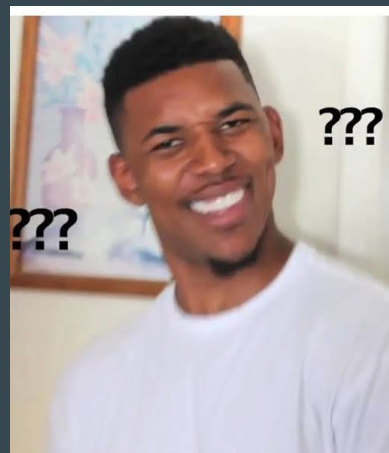


Comportamiento indefinido

```
#include <iostream>
```

```
int main() {  
    while (1);  
}
```

```
void unreachable() {  
    std::cout << "Hello, world!\n";  
}
```



Terminal

```
$ clang15++ -o main -std=c++17 -O1 -Wall -Wextra main.cpp  
$ ./main  
Hello, world!
```



Comportamiento indefinido

main es simplemente un label
seguido por unreachable()

main:

unreachable():

```
    mov     rdi, qword ptr [rip + std::cout@GOTPCREL]
    lea     rsi, [rip + .L.str]
    mov     edx, 14
    jmp     std::basic_ostream<char, std::char_traits<char>>& std::__ostream_insert<char,
std::char_traits<char>>(std::basic_ostream<char, std::char_traits<char>>&, char const*, long)@PLT
```

_GLOBAL__sub_I_example.cpp:

```
    push    rbx
    lea     rbx, [rip + std::__ioinit]
    mov     rdi, rbx
    call    std::ios_base::Init::Init()@PLT
    mov     rdi, qword ptr [rip + std::ios_base::Init::~Init()@GOTPCREL]
    lea     rdx, [rip + __dso_handle]
    mov     rsi, rbx
    pop     rbx
    jmp     __cxa_atexit@PLT
```

.L.str:

```
.asciz    "Hello, world!\n"
```



Comportamiento indefinido

```
#include <iostream>
```

```
int main() {  
    while (1);  
}
```

```
void unreachable() {  
    std::cout << "Hello, world!\n";  
}
```

```
Terminal  
$ g++ --std=c++17 -Wall -O1 ub.cpp -o main  
$ ./main
```



Comportamiento indefinido

En *safe* Rust esto no debería pasar nunca, dado que está garantizado desde el diseño del lenguaje la ausencia de comportamiento indefinido.



Concurrencia

...

C, C++

Concurrencia

```
#include <stdio.h>
#include <pthread.h>

// Shared data between threads
int counter = 0;

// The function each thread will run
void* increment_task() {
    for (int j = 0; j < 1000; j++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[16];

    for (int i = 0; i < 16; i++) {
        if (pthread_create(&threads[i], NULL, increment_task, NULL) != 0) {
            printf("Error creating thread %d\n", i);
        }
    }

    for (int i = 0; i < 16; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter: %d\n", counter);

    return 0;
}
```



Concurrencia

Memoria compartida

```
#include <stdio.h>
#include <pthread.h>

// Shared data between threads
int counter = 0;

// The function each thread will run
void* increment_task() {
    for (int j = 0; j < 1000; j++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[16];

    for (int i = 0; i < 16; i++) {
        if (pthread_create(&threads[i], NULL, increment_task, NULL) != 0) {
            printf("Error creating thread %d\n", i);
        }
    }

    for (int i = 0; i < 16; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter: %d\n", counter);

    return 0;
}
```



Concurrencia

Memoria compartida

Aumentamos el contador
mil veces

```
#include <stdio.h>
#include <pthread.h>

// Shared data between threads
int counter = 0;

// The function each thread will run
void* increment_task() {
    for (int j = 0; j < 1000; j++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[16];

    for (int i = 0; i < 16; i++) {
        if (pthread_create(&threads[i], NULL, increment_task, NULL) != 0) {
            printf("Error creating thread %d\n", i);
        }
    }

    for (int i = 0; i < 16; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter: %d\n", counter);

    return 0;
}
```



Concurrencia

Memoria compartida

```
#include <stdio.h>
#include <pthread.h>
```

```
// Shared data between threads
int counter = 0;
```

```
// The function each thread will run
void* increment_task() {
    for (int j = 0; j < 1000; j++) {
        counter++;
    }
    return NULL;
}
```

```
int main() {
    pthread_t threads[16];
```

```
    for (int i = 0; i < 16; i++) {
        if (pthread_create(&threads[i], NULL, increment_task, NULL) != 0) {
            printf("Error creating thread %d\n", i);
        }
    }
```

```
    for (int i = 0; i < 16; i++) {
        pthread_join(threads[i], NULL);
    }
```

```
    printf("Final counter: %d\n", counter);
```

```
    return 0;
}
```

Aumentamos el contador
mil veces

Lanzamos 16 hilos de
sistema para ejecutar
estos incrementos



Concurrencia

```
#include <stdio.h>
#include <pthread.h>

// Shared data between threads
int counter = 0;

// The function each thread will run
void* increment_task() {
    for (int j = 0; j < 1000; j++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[16];

    for (int i = 0; i < 16; i++) {
        if (pthread_create(&threads[i], NULL, increment_task, i)) {
            printf("Error creating thread %d\n", i);
        }
    }

    for (int i = 0; i < 16; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter: %d\n", counter);

    return 0;
}
```

```
Terminal

$ gcc -Wall -Wextra -lpthread concurrency.c -o concurrency

$ ./concurrency
Final counter: 15830

$ ./concurrency
Final counter: 15132

$ ./concurrency
Final counter: 15826

$ ./concurrency
Final counter: 14779
```



Concurrencia

```
fn main() {  
    let mut counter = 0;  
  
    std::thread::scope(|s| {  
        for _ in 0..16 {  
            s.spawn(|| {  
                for _ in 0..1000 {  
                    counter += 1;  
                }  
            });  
        }  
    });  
}
```



Concurrency

```
fn main()
{
    let mut counter = 0;
    std::thread::scope(|s| {
        for _ in 0..16 {
            s.spawn(|| {
                for _ in 0..1000 {
                    counter += 1;
                }
            });
        }
    });
}
```

error[E0499]: cannot borrow `counter` as mutable more than once at a time
--> src/main.rs:20:21

18 | std::thread::scope(|s| {
 | - has type `&'1 Scope<'1, '\_>`
19 | for _ in 0..16 {
20 | s.spawn(|| {
 | ^^^^^^^^^^^ `counter` was mutably borrowed here in the previous iteration of
the loop
 | -----
 | counter += 1;
21 | for _ in 0..1000 {
22 | counter += 1;
 | ----- borrows occur due to use of `counter` in closure
23 | }
24 | });
 | |----- argument requires that `counter` is borrowed for `'\_1`
note: requirement that the value outlives `'\_1` introduced here
--> /rustc/4a4ef493e3a1488c6e321570238084b38948f6db/library/std/src/thread/scoped.rs:200:34

For more information about this error, try `rustc --explain E0499`.



(Por qué|Quién|Dónde) Rust

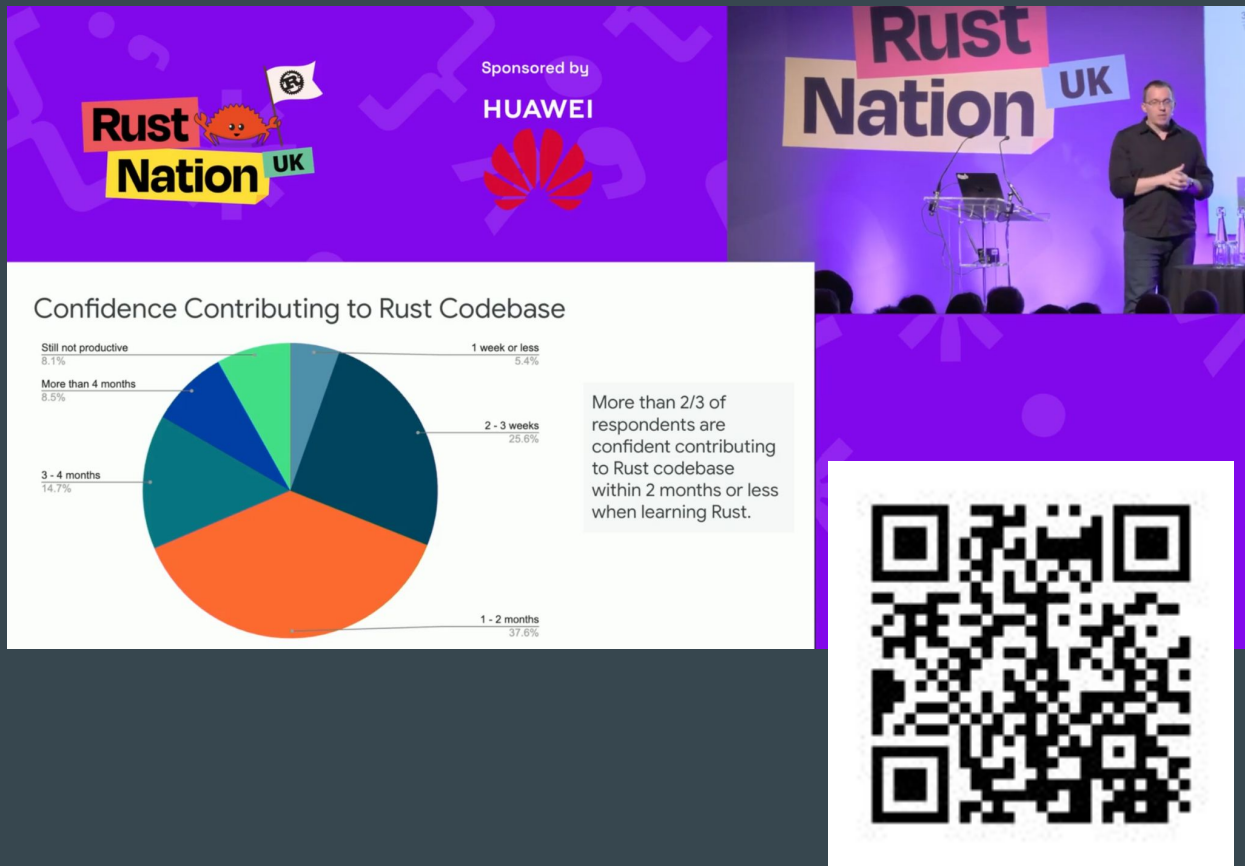
...

Las industrias y actores que ya lo adoptaron.



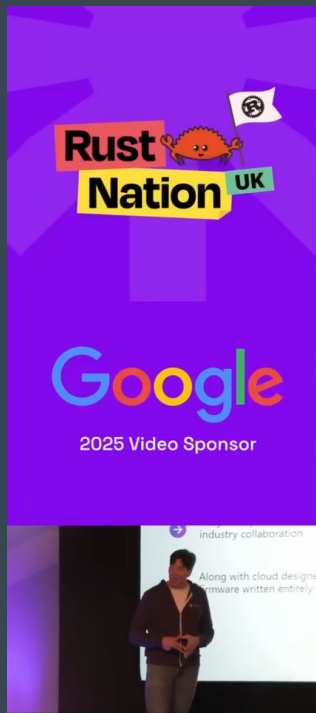
Lars Bergstrom
@ Google

Los equipos que utilizan RUST son tan productivos como los que usan Golang, y más del doble de productivos que los equipos que utilizan C++.



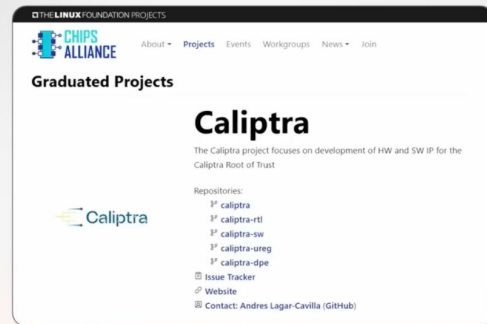
Mark Russinovich
@ Microsoft

Es hora de dejar de
iniciar nuevos
proyectos en C/C++ y
usar Rust para aquellos
casos en los que se
requiera un lenguaje
sin recolector de
basura.



Caliptra – Hardware Root of Trust

- Foundation of building more advanced security capabilities
- Fully transparent with industry collaboration
- Along with cloud designed firmware written entirely in Rust

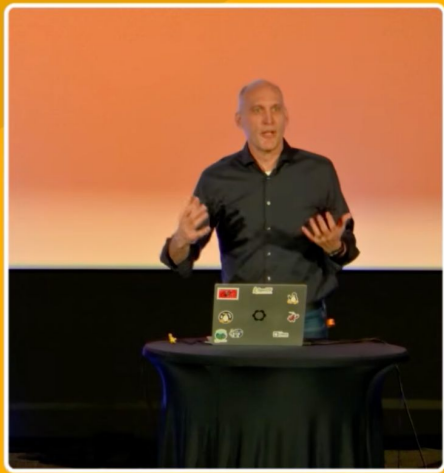


Greg Kroah-Hartman @ Linux Foundation

No me conviertas en un mantenedor que tenga que leer tu código y decir: "Ah, entonces verificas correctamente ese valor de error. ¿Obtuviste los bloqueos en el lugar correcto?". Rust nos da eso gratis. Esto es lo mejor que existe.

Rust can make reviewing code easier

- › Enforce validation of untrusted data
- › Enforce memory lifetime rules
- › Enforce locking rules
- › Enforce error handling
- › Enforce type safety



Jon Seager
@ Ubuntu

La resiliencia es
inherente al software y
a la mentalidad de la
comunidad Rust.

Rust Nation UK

arm
2026 Video Sponsor

Keyrock
2026 Platinum Sponsor

coreutils
The commands you use everyday: ls, cp, etc. Production ready!

findutils
Finding what you need: find, locate, updatedb & xargs.

diffutils
Comparing text and files: diff, cmp, diff3, sdiff.

utils.github.io

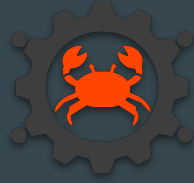


(Por qué|Quién|Dónde) Rust

...

Las comunidades que crecen en Latino America.





Oxidar



Espacio Latinoamericano para la divulgación del
lenguaje de programación Rust.





Rust Colombia



Rust México



Rust Perú



Rust Uruguay





Discord



Comunidad de habla Hispana.



Rust Programming Language
Community

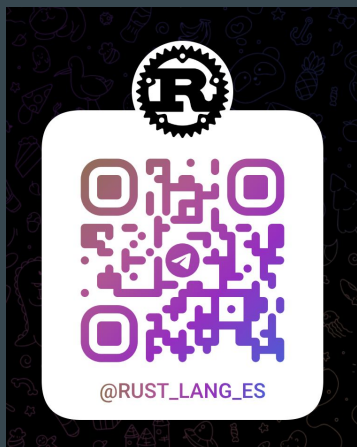




Telegram



Rust en Español



Programación en Rust



¿Preguntas?

...

¡Gracias!

